

# 3 – Linguagens Regulares

- 3.1 Sistema de Estados Finitos
- 3.2 Composição Seqüencial, Concorrente e Não-Determinista
- 3.3 Autômato Finito
- 3.4 Autômato Finito Não-Determinístico
- 3.5 Autômato Finito com Movimentos Vazios
- 3.6 Expressão Regular
- 3.7 Gramática Regular

## 3.5 Autômato Finito com Movimentos Vazios

### ◆ Movimentos vazios

- generalizam os movimentos não-determinísticos

### ◆ Movimento vazio

- transição sem leitura de símbolo algum da fita
- interpretado como um não-determinismo interno ao autômato
  - \* transição encapsulada
  - \* excetuando-se por uma eventual mudança de estados
  - \* nada mais pode ser observado

## ◆ Algumas vantagens

- facilita algumas construções e demonstrações

## ◆ Poder computacional p/ autômatos finitos

- não aumenta o poder de reconhecimento de linguagens
- qualquer  $\text{AFN}_\epsilon$  pode ser simulado por um AFD

## Def: Autômato Finito com Movimentos Vazios - $AFN_{\epsilon}$

$$M = (\Sigma, Q, \delta, q_0, F)$$

- $\Sigma$  - alfabeto (de símbolos) de entrada
- $Q$  - conjunto de estados possíveis
- $\delta$  - (função) programa ou função de transição (função parcial)

$$\delta: Q \times (\Sigma \cup \{ \epsilon \}) \rightarrow 2^Q$$

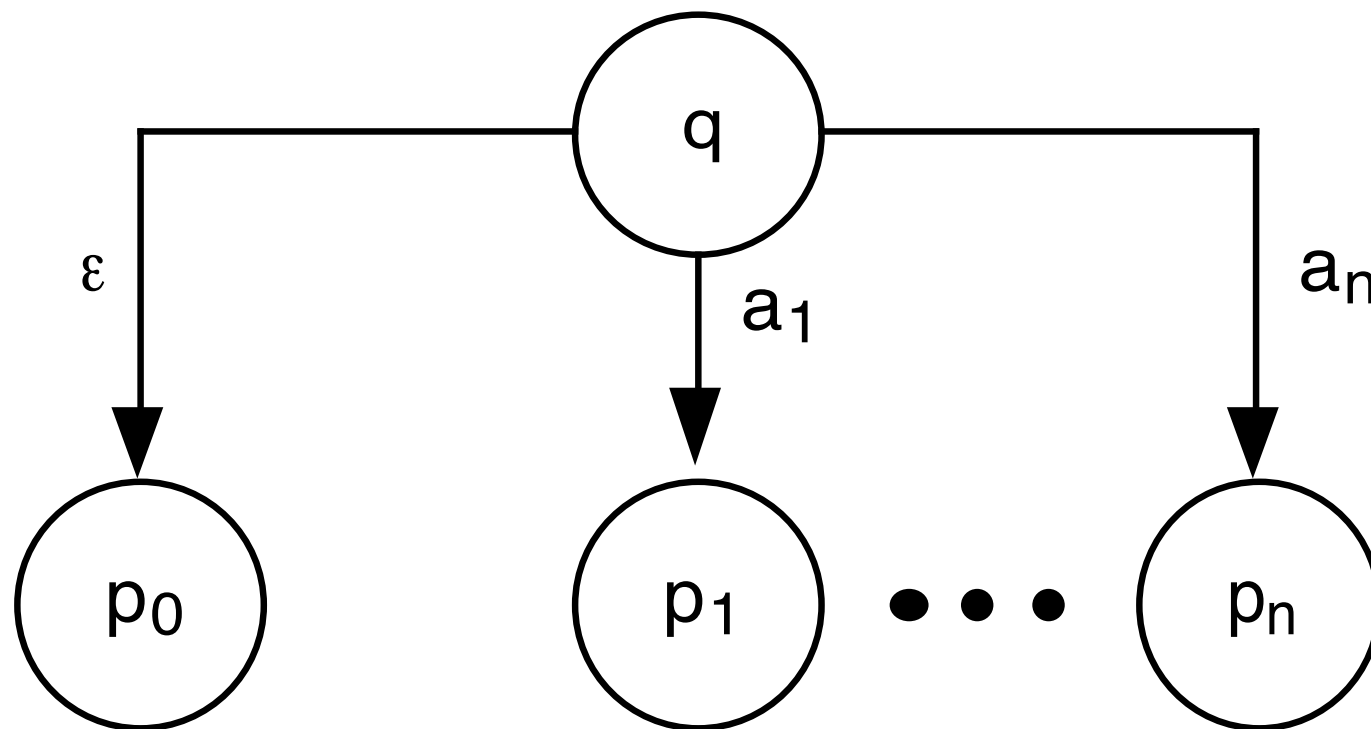
\* movimento vazio ou transição vazia

$$\delta(p, \epsilon) = \{ q_1, q_2, \dots, q_n \}$$

- $q_0$  - elemento distinguido de  $Q$ : estado inicial
- $F$  - subconjunto de  $Q$ : conjunto de estados finais

## ♦ Autômato como diagrama

$$\delta(q, \epsilon) = \{ p_0 \} \quad \delta(q, a_1) = \{ p_1 \} \quad \dots \quad \delta(q, a_n) = \{ p_n \}$$



## ◆ Computação de um $\text{AFN}_\epsilon$

- análoga à de um  $\text{AFN}$

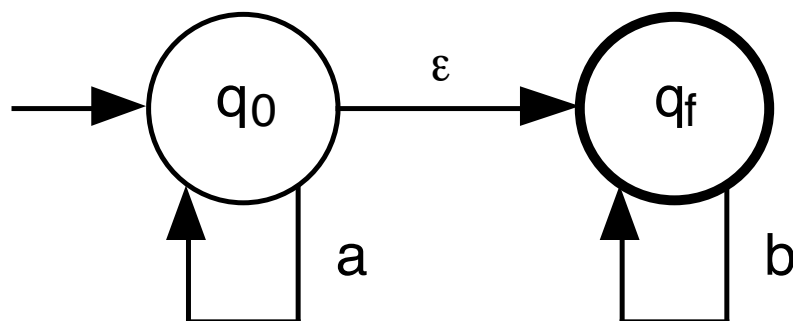
## ◆ Processamento de uma transição vazia

- não-determinístico
- assume simultaneamente os estados destino e origem
- origem de um movimento vazio: caminho alternativo

## Exp: $AFN_{\epsilon}$ : a's antecedem b's

$$M_7 = (\{a, b\}, \{q_0, q_f\}, \delta_7, q_0, \{q_f\})$$

| $\delta_7$ | a         | b         | $\epsilon$ |
|------------|-----------|-----------|------------|
| $q_0$      | $\{q_0\}$ | -         | $\{q_f\}$  |
| $q_f$      | -         | $\{q_f\}$ |            |



## ◆ Antes de definir computação

- computação de transições vazias a partir de
  - \* um estado
  - \* um conjunto finito de estados



## Def: Computação Vazia

$$M = (\Sigma, Q, \delta, q_0, F)$$

Computação Vazia ou Função Fecho Vazio (*um estado*)

$$\delta_\epsilon: Q \rightarrow 2^Q$$

- indutivamente definida
  - \*  $\delta_\epsilon(q) = \{ q \}$ , se  $\delta(q, \epsilon)$  é indefinida
  - \*  $\delta_\epsilon(q) = \{ q \} \cup \delta(q, \epsilon) \cup (\bigcup_{p \in \delta(q, \epsilon)} \delta_\epsilon(p))$ , caso contrário

Computação Vazia ou Função Fecho Vazio (*conjunto de estados*)

$$\delta_\epsilon^*: 2^Q \rightarrow 2^Q$$

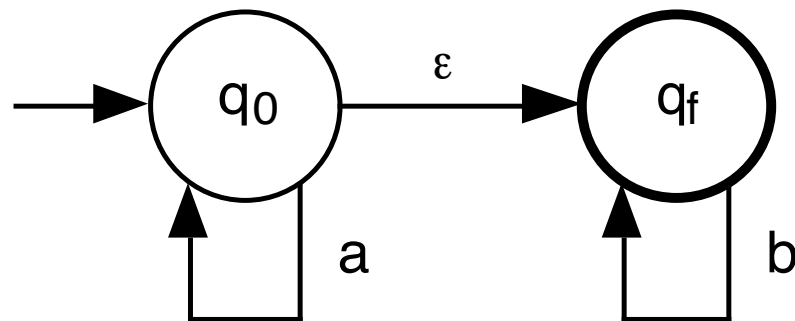
- tal que

$$\delta_\epsilon^*(P) = \bigcup_{q \in P} \delta_\epsilon(q)$$

◆ Por simplicidade,  $\delta_\epsilon$  e  $\delta_\epsilon^*$

- ambas denotadas por  $\delta_\epsilon$

## Exp: Computação Vazia



- $\delta_\epsilon(q_0) = \{ q_0, q_f \}$
- $\delta_\epsilon(q_f) = \{ q_f \}$
- $\delta_\epsilon(\{ q_0, q_f \}) = \{ q_0, q_f \}$

## ◆ Computação de um $AFN_\epsilon$ para uma entrada $w$

- sucessiva aplicação da função programa
- para cada símbolo de  $w$  (da esquerda para a direita)
- cada passo de aplicação *intercalado* com computações vazias
- até ocorrer uma condição de parada

## ◆ Assim, antes de processar a próxima transição

- determinar
  - \* todos os demais estados atingíveis
  - \* exclusivamente por movimentos vazios

## Def: Função Programa Estendida, Computação

$M = (\Sigma, Q, \delta, q_0, F)$   $AFN_\epsilon$

$$\delta^*: 2^Q \times \Sigma^* \rightarrow 2^Q$$

indutivamente definida

- $\delta^*(P, \epsilon) = \delta_\epsilon(P)$
- $\delta^*(P, wa) = \delta_\epsilon(R)$  onde  $R = \{ r \mid r \in \delta(s, a) \text{ e } s \in \delta^*(P, w) \}$

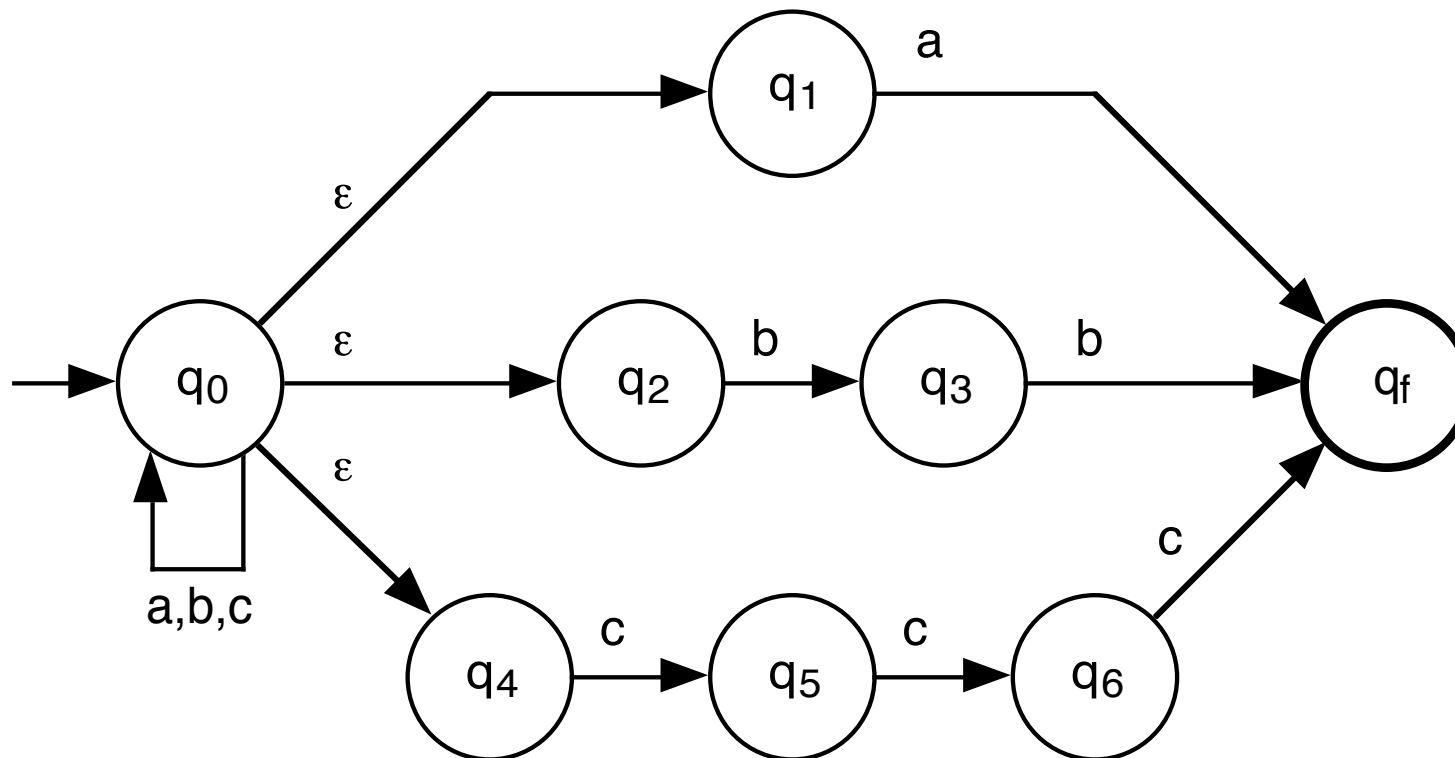
### ◆ Parada do processamento, Ling. Aceita/Rejeitada

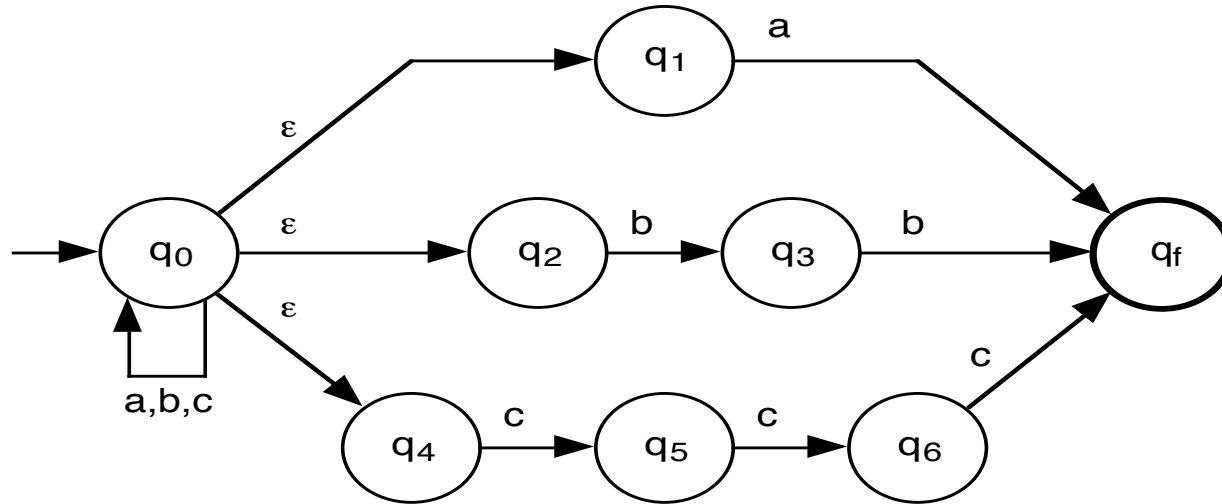
- análoga à do autômato finito não-determinístico

## Exp: Computação Vazia, Computação

$L_8 = \{ w \mid w \text{ possui como sufixo } a \text{ ou } bb \text{ ou } ccc \}$

$M_8 = (\{ a, b, c \}, \{ q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_f \}, \delta_8, q_0, \{ q_f \})$





$$\delta^*({q_0}, abb) = \delta\epsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \delta^*({q_0}, ab) \}) \quad (1)$$

$$\delta^*({q_0}, ab) = \delta\epsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \delta^*({q_0}, a) \}) \quad (2)$$

$$\delta^*({q_0}, a) = \delta\epsilon(\{ r \mid r \in \delta(s, a) \text{ e } s \in \delta^*({q_0}, \epsilon) \}) \quad (3)$$

Como:

$$\delta^*({q_0}, \epsilon) = \delta\epsilon({q_0}) = \{ q_0, q_1, q_2, q_4 \} \quad \text{considerado em (3)}$$

$$\delta^*({q_0}, a) = \{ q_0, q_1, q_2, q_4, q_f \} \quad \text{considerado em (2)}$$

$$\delta^*({q_0}, ab) = \{ q_0, q_1, q_2, q_3, q_4 \} \quad \text{considerado em (1)}$$

Resulta na computação:  $\delta^*({q_0}, abb) = \{ q_0, q_1, q_2, q_3, q_4, q_f \}$

## Teorema: Equivalência entre AFN e $AFN_\epsilon$

Classe dos Autômatos Finitos com Movimentos Vazios é equivalente à Classe dos Autômatos Finitos Não-Determinísticos

Prova: (*por indução*)

Mostrar que

- a partir de um  $AFN_\epsilon$   $M$  qualquer
- construir um  $AFN$   $M_N$  que realiza as mesmas computações
- $M_N$  simula  $M$

$AFN_\epsilon \rightarrow AFN$

- construção de uma função programa sem movimentos vazios
- conjunto de estados destino de cada transição não-vazia
  - \* ampliado com os demais estados possíveis de serem atingidos exclusivamente por transições vazias

$M = (\Sigma, Q, \delta, q_0, F)$  um  $AFN_\epsilon$  qualquer.  $AFN$  construído

$$M_N = (\Sigma, Q, \delta_N, q_0, F_N)$$

- $\delta_N: Q \times \Sigma \rightarrow 2^Q$  é tal que

$$\delta_N(q, a) = \delta^*(\{q\}, a)$$

- $F_N$  é o conjunto de todos os estados  $q$  pertencentes a  $Q$

$$\delta_\epsilon(q) \cap F \neq \emptyset$$

\* estados que atingem estados finais via computações vazias

Demonstração que, de fato, o  $AFN M_N$  simula o  $AFN_\epsilon M$

- indução no tamanho da palavra
- exercício



◆ Portanto, linguagem aceita por  $AFN_{\epsilon}$

- é Linguagem Regular ou Tipo 3